

Music Genre Classification using Neural Networks

- **Author** : Yashasvi Bassi
 - **Roll Number** : 23f2003131
 - **University** : Indian Institute of Technology Madras (IIT Madras)
 - **Email** : 23f2003131@ds.study.iitm.ac.in
 - **Course** : BSDA2001P — Introduction to Deep Learning and Generative AI
 - **Deployed at** : https://huggingface.co/spaces/Yashasvi8007/music_genre_clf
-

Introduction

Music genre classification is an important task in music information retrieval and audio understanding. Automatic classification systems help organize large music collections, improve recommendation systems, and support music analytics. However, the task becomes significantly more challenging when dealing with **mashups**, where multiple musical tracks are mixed together. Mashups combine different musical stems such as **drums, bass, vocals, and other instruments**. These mixtures introduce overlapping spectral patterns and complex audio interactions, making genre classification more difficult than analyzing isolated songs.

This project presents a **deep learning approach for genre classification of mashup audio** using synthetic training data generated from song stems. By randomly mixing stems from songs belonging to the same genre, the model learns to recognize genre characteristics even when instruments are partially missing or mixed differently.

The system converts audio signals into **mel spectrogram representations** and trains a **convolutional neural network (CNN)** based on the various architectures discussed in the later parts of this report. The model is evaluated using the **macro F1 score**, and it achieved the best **validation F1 score of 0.9114** and a **Kaggle leaderboard score of 0.867**.

Dataset Description

The dataset consists of music tracks organized by genre. Each song is separated into four audio stems representing different instrumental components. The four stems used are:

- **Drums**
- **Bass**
- **Vocals**
- **Other instruments**

Each genre folder contains multiple songs, and each song contains these four isolated stems. This structure enables flexible recombination of stems to simulate realistic mashups during training.

Dataset Structure

```
genres_stems/  
  genre_name/  
    song_name/  
      — drums.wav  
      — bass.wav  
      — vocals.wav  
      — other.wav
```

Additionally, a **test dataset** contains mashup audio clips where multiple sources are mixed. The goal of the model is to predict the **dominant genre** of each mashup track.

Audio Settings

All audio files are processed using the following configuration:

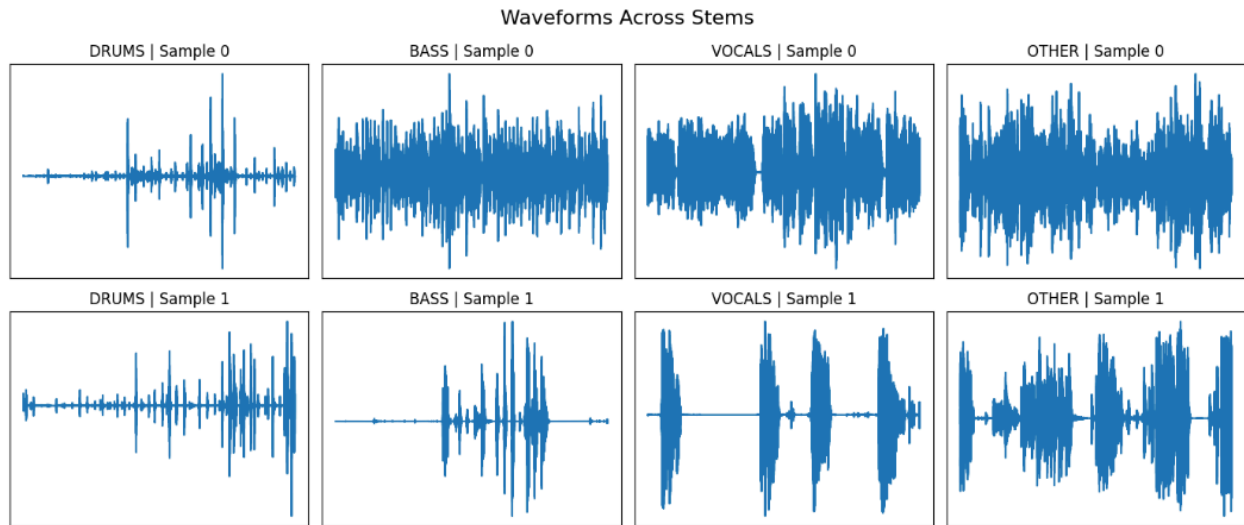
Parameter	Value
Sampling rate	22050 Hz
Clip duration	7 seconds
Samples per clip	154,350
Mel frequency bins	256

Standardizing these parameters ensures consistent input dimensions for the neural network.

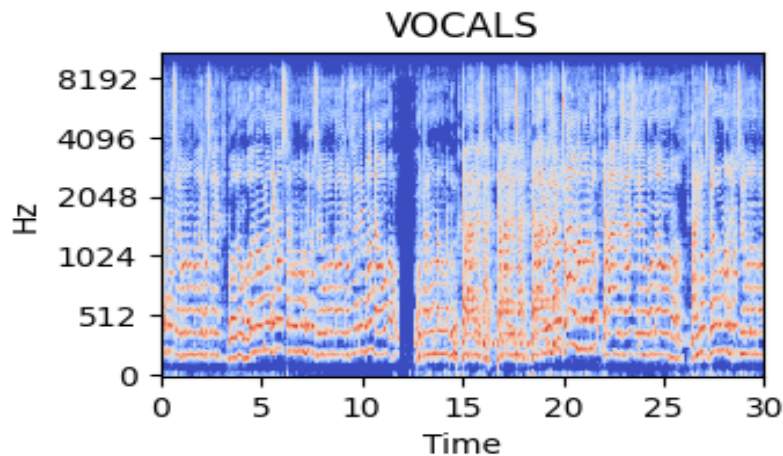
Exploratory Data Analysis

Exploratory data analysis (EDA) was conducted across three complementary representations to understand the audio signal characteristics:

Waveform Visualization: Time-domain analysis of audio signals, showing amplitude variations over time. This provided initial insights into signal length, volume dynamics, and the structural differences between instrument stems.



Spectrogram Analysis: Frequency-domain visualization showing spectral content across different time frames. Spectrograms revealed genre-specific frequency patterns — for example, the heavy low-frequency presence in bass-dominated genres such as hiphop and metal.



Data Preprocessing

Before training, all audio signals undergo preprocessing to transform raw waveforms into suitable machine learning features.

Audio Cropping

Audio clips may vary in length, so a **fixed duration of 7 seconds** is used for training. Each audio signal is randomly cropped to obtain a segment of this length. If the signal is shorter than 7 seconds, **zero padding** is applied to maintain consistent input size.

Random cropping also acts as a **data augmentation technique**, allowing the model to observe different parts of the same track.

Synthetic Mashup Generation

A key aspect of the approach is **synthetic mashup creation**. Instead of training only on full songs, the system generates new training samples by combining different stems.

Possible combinations include:

- All four stems together
- Two-stem combinations (e.g., drums + bass)
- Three-stem combinations
- Randomly selected subsets of stems

This introduces variability and improves generalization.

Data Augmentation via Stem Mixing

A key innovation in this project is the creation of synthetic training samples through dynamic stem mixing.

For each song multiple combinations of stems are randomly selected (e.g., drums + vocals, bass + other, etc.). Signals are summed and normalized

Additionally random combinations of 2–4 stems are generated dynamically. Each training sample is augmented 16 times (`TRAIN_MULTIPLIER = 16`) **This significantly increases dataset diversity and simulates real-world mashups.**

4. Feature Extraction

Raw audio signals are converted into **mel spectrograms**, which represent the distribution of energy across frequencies over time.

Mel spectrograms are widely used in audio classification because they approximate human auditory perception.

The spectrogram is computed using the Short-Time Fourier Transform (STFT) and then mapped to the mel scale.

Mel Spectrogram Parameters

Parameter	Value
FFT size	2048

Hop length	512
Mel bands	256
Minimum frequency	10 Hz
Maximum frequency	11025 (SR//2)

The resulting spectrogram is converted into **decibel scale** and then standardized using mean and variance normalization. This normalization ensures that the model focuses on **relative spectral patterns** rather than absolute energy levels.

Data Augmentation

To improve generalization and prevent overfitting, additional data augmentation techniques are applied.

Training Mix Expansion

Each song produces **16 different synthetic mixes** using various stem combinations and gain adjustments. This dramatically increases the diversity of training samples and exposes the model to different instrument arrangements.

Spectrogram Augmentation (SpecAugment)

SpecAugment is applied during training directly on the spectrogram representation. Two types of masking are used:

Frequency Masking Random frequency bands are masked by setting them to zero. This forces the model to rely on other spectral features.

Time Masking Random time segments of the spectrogram are masked, encouraging the model to learn more robust temporal patterns. These augmentations help the model handle missing information and noisy signals.

Dataset Preparation

Each mel spectrogram is converted into a tensor suitable for input to the neural network. Since convolutional neural networks designed for images expect **three input channels**, the single-channel spectrogram is replicated three times. The spectrogram is also resized to **224 × 224 pixels**, which matches the expected input size of the EfficientNet architecture. The dataset is split into:

- **90% training data**
- **10% validation data**

The split is done using **stratified sampling** to ensure equal genre distribution across both sets.

Model Architectures

Model 1 “Cnn from scratch”

The proposed model follows a convolutional neural network design composed of four convolutional blocks, followed by a classifier head. This architecture is structured to progressively extract hierarchical features from the input data and map them to the final genre classification output.

Block 1

The first block transforms the input from 1 channel to 32 channels using a single convolutional layer with a 3×3 kernel and padding of 1. This is followed by Batch Normalization to stabilize training and a ReLU activation function to introduce non-linearity. A max-pooling layer with a 2×2 window is applied to reduce spatial dimensions and control computational complexity.

Block 2

The second block increases the feature depth from 32 to 64 channels using a convolutional layer with similar kernel settings. Batch Normalization and ReLU activation are applied, followed by max-pooling to further downsample feature maps and improve translational invariance.

Block 3

The third block expands the representation from 64 to 128 channels using a convolutional layer. Batch Normalization and ReLU activation are applied to enhance feature stability and non-linearity. Max-pooling is again used for spatial reduction. This block forms the final stage of hierarchical feature extraction.

After the convolutional blocks, Adaptive Average Pooling is applied to reduce each feature map to a fixed size of 1×1, converting the output into a compact 128-dimensional feature vector. This ensures that the model is independent of input image size while significantly reducing the number of parameters and improving computational efficiency. The classification head consists of a single fully connected layer that maps this 128-dimensional feature vector directly to the number of target classes (genres), producing the final output logits. This minimal design removes unnecessary complexity while maintaining strong discriminative capability.

The model uses the ReLU activation function, defined as $\text{ReLU}(x) = \max(0, x)$, which is computationally efficient and helps mitigate the vanishing gradient problem, making it well-suited

for this lightweight architecture. Batch Normalization is applied after each convolutional layer to stabilize training and accelerate convergence. Since the model is already simplified and uses global average pooling, dropout is not heavily required, as the reduced parameter count inherently lowers the risk of overfitting. Additionally, label smoothing with a value of 0.1 is used in the CrossEntropyLoss function to prevent the model from becoming overly confident in its predictions and to improve generalization performance.

Training config for the CNN

```
batch_size:64
duration:7
epochs:32
lr:0.00032
n_mels:256
optimizer:"AdamW"
scheduler:"CosineAnnealingLR"
train_multiplier:14
best_val_f1:0.8003552582533603
```

WANDB run : [customcnn_run](#)

Model 2 - EfficientNet-B0 Architecture

EfficientNet-B0 is a convolutional neural network designed to achieve high performance with improved computational efficiency. Unlike traditional architectures that scale depth or width independently, EfficientNet uses a **compound scaling method** that uniformly scales depth, width, and input resolution, resulting in better accuracy with fewer parameters.

The architecture is built using **MBConv (Mobile Inverted Bottleneck Convolution) blocks**, which are derived from mobile-efficient networks. Each block consists of depthwise separable convolutions, squeeze-and-excitation (SE) modules, and skip connections. These components allow the model to capture important features while keeping computation and memory usage low. The use of SE blocks further improves performance by adaptively recalibrating channel-wise feature responses.

EfficientNet-B0 serves as the baseline model in the EfficientNet family and is relatively lightweight while maintaining strong feature extraction capabilities. It includes multiple stages of MBConv blocks with varying expansion ratios and kernel sizes, enabling it to learn both local and global patterns effectively.

In this project, EfficientNet-B0 is adapted for audio classification by converting audio signals into **mel spectrograms**, which provide a 2D time-frequency representation similar to images. This allows the use of pretrained ImageNet weights for transfer learning. The pretrained model helps

in extracting general visual patterns that transfer well to spectrogram data, improving convergence and performance on limited datasets.

The final classification layer is modified to match the number of target classes (music genres), and input spectrograms are resized and converted to three channels to be compatible with the pretrained architecture.

EfficientNet-B0 offers advantages such as high accuracy with fewer parameters, efficient computation, and strong performance with transfer learning. However, similar to other image-based models, it faces limitations such as domain mismatch between natural images and audio spectrograms, and possible information loss due to resizing.

Overall, EfficientNet-B0 provides an efficient and scalable architecture, making it well-suited for this audio classification task while maintaining a balance between performance and computational cost.

Training config for EfficientNet-B0

```
batch_size:48
duration:7
epochs:25
learning_rate:0.00032
model:"efficientnet_b0"
n_mels:256
optimizer:"AdamW"
scheduler:"CosineAnnealingLR"
Train_multiplier:16
best_val_f1:0.9114475582896636
```

WANDB run : [efficientnetb0_run](#)

Model 3 - ResNet-50 Architecture

ResNet-50 is a deep convolutional neural network designed to address the degradation and vanishing gradient problems in very deep models. It achieves this through **residual learning**, where shortcut (skip) connections allow inputs to bypass intermediate layers. Instead of learning a direct mapping $F(x)F(x)F(x)$, the network learns a residual function $F(x)+xF(x) + xF(x)+x$, which improves gradient flow and stabilizes training.

The architecture consists of 50 layers, including convolution, batch normalization, and ReLU activation layers. It is organized into four stages of residual blocks (3, 4, 6, and 3 blocks respectively). Each block uses a **bottleneck structure** of 1×1 , 3×3 , and 1×1 convolutions, which reduces computational cost while preserving strong feature extraction capability. Shortcut

connections are either identity mappings or projection layers (1×1 convolutions) to handle changes in dimensions.

In this project, ResNet-50 is adapted for audio classification by converting audio signals into **mel spectrograms**, which provide a 2D time-frequency representation similar to images. This allows the use of convolutional networks pretrained on ImageNet. Transfer learning helps the model leverage learned low-level features, improving performance and convergence on limited data.

The final fully connected layer is modified to match the number of target classes (music genres), and spectrograms are resized and converted to three channels for compatibility with the pretrained model.

ResNet-50 offers strong feature extraction and efficient training of deep networks. However, it has limitations such as domain mismatch between image and audio data, and potential information loss due to resizing of spectrograms.

Overall, ResNet-50 provides a reliable and effective architecture for this audio classification task due to its depth, residual connections, and compatibility with transfer learning.

Training config for Resnet

```
batch_size:48
duration:7
epochs:32
learning_rate:0.0005
optimizer:"AdamW"
sample_rate:22,050
scheduler:"CosineAnnealingLR"
train_multiplier:16
Weight_decay:0.00008
Best_val_f1: 0.8679015721120985
WANDB run:Resnet\_run
```

Learning Rate Scheduling

For all the models used above, **cosine annealing learning rate scheduler** is used to gradually reduce the learning rate during training. This schedule allows the model to explore the parameter space initially and then converge smoothly toward an optimal solution. Gradient clipping is also applied to stabilize training and prevent exploding gradients.

$$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min}) \left(1 + \cos \left(\frac{T_{cur}}{T_{max}} \pi \right) \right)$$

Cross-Entropy Loss

In the context of this problem music genre classification, **Cross-Entropy Loss** (Log Loss) serves as the optimization objective that measures the performance of the deep learning model. It quantifies the disparity between the model's predicted probability distribution and the actual labels. For multi-class tasks, the model typically outputs a vector of "logits" via a Softmax function, which ensures the predicted probabilities for all genres (e.g. Jazz, Metal, Pop) sum to one.

The loss is calculated by taking the negative sum of the log of the predicted probabilities, weighted by the true labels:

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Because the loss function is logarithmic, it heavily penalizes "confident" incorrect predictions. If the model assigns a low probability to the true genre, the loss value increases exponentially.

Model Evaluation

Model performance is measured using the **macro F1 score**, which considers both precision and recall for each genre. Macro averaging treats all classes equally, making it suitable for datasets where class imbalance may exist. During validation, predictions are collected across the validation dataset and compared with the ground truth labels to compute the final F1 score.

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Test Inference

Test audio files may be longer than the training clips. To handle this, each test audio is divided into **7-second segments**.

Each segment undergoes the same preprocessing steps used during training.

The model predicts genre probabilities for each segment individually. The final prediction for the entire audio file is obtained by **averaging the probabilities across all segments**.

This approach ensures that the model considers information from the entire track rather than a single portion.

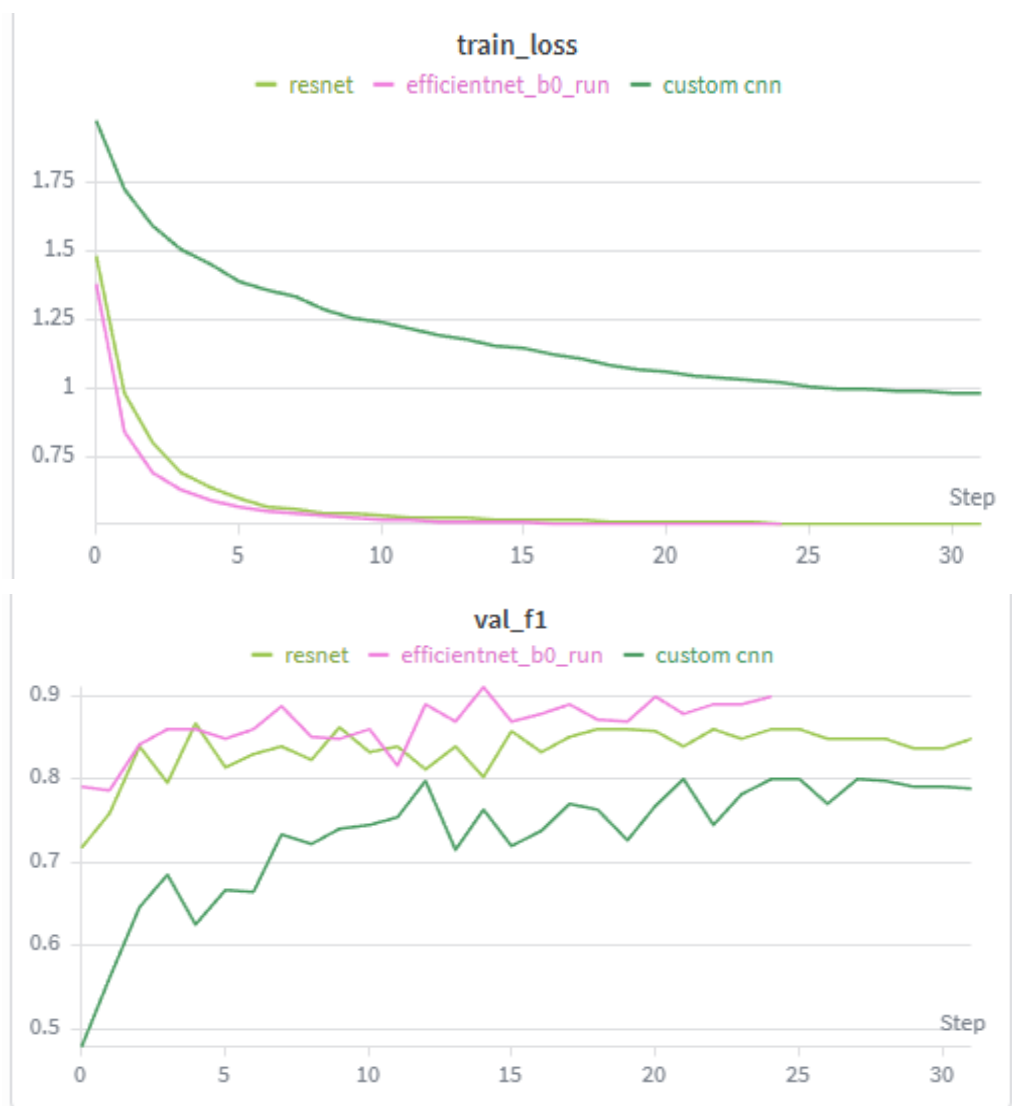
Final Results

The custom CNN exhibited a gradual and steady decrease in training loss, starting from approximately 1.97 and reaching around 0.98 by the end of training. This slow convergence indicates that the model was able to learn meaningful patterns from the data, but at a limited pace. Correspondingly, the validation F1-score improved from an initial value of 0.47 to a peak of approximately 0.80 after more than 20 epochs. However, the improvement plateaued beyond this point, suggesting that the model lacked sufficient capacity to capture more complex features in the data. This behavior is indicative of underfitting, where the model is unable to fully represent the underlying data distribution despite prolonged training.

In contrast, the ResNet model demonstrates significantly faster convergence. The training loss drops sharply from 1.48 to below 0.80 by the third epoch and stabilizes around 0.50 in subsequent epochs. The validation F1-score shows strong initial performance, reaching 0.71 in the first epoch and peaking at 0.8679 as early as epoch 5. Beyond this point, the F1-score fluctuates between 0.83 and 0.86, indicating that no further substantial improvements occur in later epochs. This pattern suggests mild overfitting, where the model begins to specialize to the training data after achieving its optimal performance early in the training process.

Among the three models, EfficientNet-B0 delivered the best overall performance. Similar to ResNet, it exhibited rapid convergence, with training loss decreasing from 1.38 to approximately 0.69 within the first three epochs and stabilizing near 0.50 in later stages. However, unlike ResNet, EfficientNet continued to improve its validation performance over a longer period. The validation F1-score increased steadily, reaching 0.88 by epoch 8 and achieving a maximum value of 0.9114 at epoch 15. Even beyond this peak, the model maintained stable performance within the range of 0.87 to 0.90, indicating strong generalization and robustness. This sustained improvement suggests that EfficientNet-B0 is better able to learn rich feature representations without overfitting prematurely.

A comparative analysis reveals clear differences in model performance. The custom CNN underperforms due to limited capacity, while ResNet converges quickly but shows slight overfitting after early peak performance. EfficientNet-B0 achieves the highest validation F1-score (0.9114) and maintains stable performance, indicating superior generalization. Overall, the results highlight the importance of model architecture and pretrained networks. EfficientNet-B0 is the most effective model, providing the best balance of accuracy and stability, with optimal performance at epoch 15.



These results demonstrate the effectiveness of combining **synthetic mashup training, spectrogram features, and convolutional neural networks**.the final model chosen according to the results was [EfficientNet-B0](#).

Model	Validation Score	Kaggle Score
EfficientNet-B0	0.9114	0.869
Custom CNN	0.8003	0.701
ResNet-50	0.86	0.821

Challenges Faced

The project encountered several challenges during its development and implementation. One major difficulty was handling noisy and complex audio signals that contained varying environmental backgrounds, which made feature extraction and model generalization more difficult. Another significant challenge was preventing overfitting. Additionally, there was an initial lack of domain knowledge, along with insufficient use of data augmentation techniques and limited data generation through train multiplication, which impacted early model performance. Finally, managing training time efficiently while ensuring optimal GPU utilization also posed a technical constraint throughout the process.

Future Work

Advanced Augmentation: Additional augmentation techniques including time stretching, pitch shifting, and more sophisticated noise mixing strategies should be explored to further close the training-test distribution gap.

Architecture Variants: Experimenting with advanced transformer architectures such as AST, Wav2Vec 2.0, or HuBERT could yield improved audio representation learning, as these models incorporate both convolution and attention mechanisms optimized for audio.

Conclusion

This project presents a deep learning pipeline for **music genre classification from mashup audio**. The system generates synthetic training data by mixing stems from individual songs, converts audio signals into mel spectrograms, and trains an EfficientNet-based CNN to classify genres. The model successfully learns genre characteristics even when only partial musical components are present. Synthetic mixing significantly improves the model's ability to generalize to real mashup tracks. Using **EfficientNet** provides strong feature extraction capability while maintaining computational efficiency.

Data augmentation techniques such as random stem mixing and SpecAugment help the model generalize to complex mashup recordings. Segment-based inference further improves predictions for longer audio tracks. The proposed method achieved a **Kaggle score of 0.869**, demonstrating strong performance on a challenging mashup classification task.

References

Librosa Development Team. (n.d.). *Librosa: Audio and music signal analysis in Python*.
<https://librosa.org/>

Librosa Documentation. (n.d.). *Mel spectrogram*.
<https://librosa.org/doc/latest/generated/librosa.feature.melspectrogram.html>

Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking model scaling for convolutional neural networks*.
<https://arxiv.org/abs/1905.11946>

Wikipedia Contributors. (n.d.). *Transfer learning*.
https://en.wikipedia.org/wiki/Transfer_learning

Wikipedia Contributors. (n.d.). *Cross entropy*.
https://en.wikipedia.org/wiki/Cross_entropy

use of AI-based tools, for minor assistance in debugging errors, inline code completions.